lRepublic of the Philippines

Laguna State Polytechnic University

Province of Laguna

COLLEGE OF COMPUTER STUDIES

C# Console Application Final Project Proposal

Proponent/Researcher:	1. Aragones, Winslie Ghlen A. 2. Manito, Jared A. 3. Rafael, Khen Joshki D.
Project Title: (5 to 15 word count): A C# Console Application for Secure Local Password Management and Generation	
Introduction	Today, almost everyone uses multiple online accounts for school, social media, bank accounts, and personal tasks. Most people manage their credentials by memorizing them, writing them down on paper, saving them in plain text files, or using the same password across every service. These habits cause common problems: people easily forget complex passwords, written notes can get lost or exposed, and reusing the same password leaves every account open to attack if one is breached. A simple computer program solves these issues by giving users a single, organized place to keep their account details. The proposed C# console application assists users through basic data input by letting them enter account names, usernames, and passwords through clear console prompts. It can also automatically generate random, strong passwords. It also handles record management by letting users view, add, search, and delete saved accounts whenever needed. Finally, the application uses a WriteLine to show the stored accounts in a clean and readable Console. This project is relevant because it helps students and everyday computer users build safer digital options on managing accounts. The main benefit is making account management easy and stress-free, freeing users from memorizing dozens of logins while keeping their information organized.
Statement of the Problem	Managing multiple online accounts manually is inconvenient, inefficient, and risky. Relying on human memory often leads to forgotten logins, while writing credentials in physical notebooks or plain text files makes it difficult to organize, search, and update records. These traditional habits also involve repetitive tasks, such as repeatedly typing and resetting forgotten credentials, and increase the chance of recording errors like mistyping case-sensitive passwords. In addition, coming up with new, complex passwords for every single account is difficult, often leading users to choose weak or repeated passwords that compromise their personal security. To address these limitations, this project aims to create a C# Console Application that provides a central, structured tool to safely manage account records and generate random passwords.

	Specifically, this proposed console application intends to answer the following questions: 1. How can the application collect and validate user inputs, such as platform names, usernames, and passwords, through the console? 2. How can the system automatically generate strong, random passwords to eliminate weak and reused credentials? 3. How can the program organize account records so users can easily view, search, update, and delete them without errors?
Objective of the Study	**General Objective** To create a simple C# console application that lets users save their passwords and generate random ones. **Specific Objectives** ● To accept and validate user inputs, such as platform names, usernames, and passwords, using appropriate variables and data types. ● To implement conditional statements and loops to navigate through menu options and control program flow. ● To perform password generation operations by assembling random combinations of letters, numbers, and symbols based on user preference. ● To manage records by allowing users to add, search, update, and delete account details without errors. ● To display organized, easy-to-read account information and status prompts directly through the console interface. ● To test the application and verify that all core features work accurately and reliably.
Scope and Limitation of the Study:	This study focuses on the development of a C# console program for secure local password management and generation, specifically for students who handle multiple online accounts. All features of the program are created through the console. Users interact with the program through text prompts and a numbered menu, where they type their choices using the keyboard, and the program responds using Console.WriteLine and Console.ReadLine. The console accepts platform names, usernames, and passwords, and it validates each entry by displaying invalid messages when a field is empty, then asking the user to re-enter the data. The same console interface lets users generate random passwords, choosing the length and character types through simple prompts, and manage records by adding, viewing, searching, updating, and deleting accounts. Saved accounts and status messages are displayed in an organized and readable text layout The program runs in a loop that returns to the main menu after each action until the user selects the exit option. The study is limited to the design and implementation of these console features using basic C# programming constructs such as variables, console input and output, conditional statements, loops, and methods. The study is limited to a console-based program and does not include a graphical user interface, mobile version, or web version. The program is intended for local use only, meaning that records are stored on a single device without cloud storage, online syncing, or multi-device access. It is also designed for a single user on one computer and does not support multiple user accounts or shared access. The program does not offer security features such as biometric login, two-factor authentication, or enterprise-grade encryption, and it cannot match the capabilities of professional password managers such as KeePass. In addition, the application does not provide browser or app auto-fill, nor does it support importing or exporting data from other password managers. If the user forgets the application's access

	credentials, the saved data cannot be recovered. Since the study targets students, it does not address the needs of businesses, organizations, or large-scale users. Lastly, the application avoids advanced algorithms and complex technical features and relies only on basic C# programming constructs, in line with the scope of the course.
Review Literatures and System	**Password Management** Temoshok et al. (2025), in the National Institute of Standards and Technology's Digital Identity Guidelines, discuss the use of password managers to help users maintain different passwords for their online accounts. Using unique passwords helps reduce the risk of attackers accessing multiple accounts through a compromised password. This is related to the proposed system because it aims to provide users with an organized way to manage their account credentials and reduce their reliance on memorizing passwords. **Password Generation and Security** The National Institute of Standards and Technology (NIST, 2025) provides guidance on password security and the importance of using distinct passwords for different accounts. Generating random passwords can help users avoid predictable password choices. This supports the proposed C# console application, which includes a password generation feature that allows users to create random passwords using different combinations of characters. **Protection of Stored Information** The Open Worldwide Application Security Project (OWASP, n.d.) explains that sensitive information should be protected through appropriate security measures. Its Cryptographic Storage Cheat Sheet discusses the importance of selecting suitable encryption methods and protecting sensitive data stored on a device. This is relevant to the proposed application because account credentials will be stored locally. Therefore, the system should consider appropriate encryption and access protection to help prevent unauthorized access to saved information. **KeePass Password Manager** KeePass is a password management application that provides features such as password storage, searching, sorting, and random password generation (KeePass, n.d.). It allows users to organize their credentials and generate passwords according to their preferred character types and lengths. These features are related to the proposed system, which also aims to manage account records and generate random passwords. However, the proposed application will focus on a simpler, menu-driven C# console interface designed for basic local password management. **Password Management Behavior** Merdenyan and Petrie (2022), in their study *Two Studies of the Perceptions of Risk, Benefits and Likelihood of Undertaking Password Management Behaviours*, examined how people understand the risks and benefits of managing their passwords. The study discusses common practices such as reusing passwords and writing them down. This is related to the proposed

	system because users who manage several online accounts may have difficulty keeping their credentials organized. The proposed C# console application aims to provide a more organized way to store account details and generate passwords for different accounts. **Password Managers, Trust, and Security** Alodhyani et al. (2020), in their study *Password Managers—It's All about Trust and Transparency*, examined how users perceive password managers and the factors that influence their use. The researchers conducted studies involving password manager interfaces, usability testing, interviews, and questionnaires. Their findings indicated that trust and transparency were important concerns among users. This is relevant to the proposed C# console application because it highlights the importance of helping users understand how their account information is managed and protected. The proposed system will focus on providing clear menu options for managing account records and generating passwords while considering the protection of locally stored credentials. **Functions and Usability of Password Managers** Simmons et al. (2021), in their study *Systematization of Password Manager Use Cases and Design Paradigms*, examined the common functions and design approaches of password managers. The researchers identified essential, recommended, and extended use cases and conducted 136 cognitive walkthroughs involving eight password managers. Their study provides useful information about the functions users may need when managing passwords. This is related to the proposed C# console application because it includes basic account management features, such as adding, viewing, searching, updating, and deleting records, as well as generating passwords. The study can help guide the selection of practical features for a simple password management system. **Password Generation and Composition Policies** Gautam et al. (2022), in their study titled *Improving Password Generation Through the Design of a Password Composition Policy Description Language*, examined how password managers can generate passwords that meet the requirements of different websites. The researchers developed a method for describing password composition policies so that password managers could generate passwords that follow those requirements. Their study also included a user evaluation involving 25 participants. This is relevant to the proposed C# console application because it highlights the importance of generating passwords that meet specific character requirements. The proposed system will include a password generation feature that allows users to create random passwords using different combinations of characters.
References:	Alodhyani, F., Theodorakopoulos, G., & Reinecke, P. (2020). Password managers—It's all about trust and transparency. Future Internet, 12(11), Article 189. https://doi.org/10.3390/fi12110189 OWASP Foundation. (n.d.). Cryptographic storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html Reichl, D. (n.d.). Features. KeePass. https://keepass.info/features.html Merdenyan, B., & Petrie, H. (2022). Two studies of the perceptions of risk, benefits and likelihood of undertaking password management behaviours.

	Behaviour & Information Technology, 41(12), 2514–2527. https://doi.org/10.1080/0144929X.2021.2019832 Alodhyani, F., Theodorakopoulos, G., & Reinecke, P. (2020). Password managers—It's all about trust and transparency. Future Internet, 12(11), Article 189. https://doi.org/10.3390/fi12110189 Simmons, J., Diallo, O., Oesch, S., & Ruoti, S. (2021). Systematization of password manager use cases and design paradigms. In Proceedings of the 37th Annual Computer Security Applications Conference (pp. 528–540). Association for Computing Machinery. https://doi.org/10.1145/3485832.3485889 Gautam, A., Lalani, S., & Ruoti, S. (2022). Improving password generation through the design of a password composition policy description language. In Proceedings of the 18th Symposium on Usable Privacy and Security (SOUPS 2022). USENIX Association. https://www.usenix.org/conference/soups2022/presentation/gautam

Prepared by:

Winslie Ghlen A. Aragones
Jared A. Manito
Khen Joshki D. Rafael

Business and Functional Requirements

Business Requirement ID	Business Requirement Description	Functional Requirement ID	Functional Requirement Description
BR-001	Keep account login details organized in one place.	FR-001	The system shall allow the user to add an account by entering a platform name, username, and password.
		FR-002	The system shall validate that the platform name, username, and password are not empty.
		FR-003	The system shall display all saved accounts in a numbered list showing the platform, username, and password.
		FR-004	The system shall allow the user to search for accounts by platform name, ignoring uppercase and lowercase.
BR-002	Keep saved account information accurate and up to date.	FR-005	The system shall allow the user to update the username and password of a selected account.
		FR-006	The system shall validate that the selected account number is numeric and within the range of saved accounts.
		FR-007	The system shall allow the user to delete a selected account after confirming with Y or N.
BR-003	Help users create strong passwords without thinking of one themselves.	FR-008	The system shall generate a random password with a length chosen by the user from 8 to 32 characters.
		FR-009	The system shall allow the user to choose whether the password includes letters, numbers, and symbols, and shall use letters if none is chosen.
		FR-010	The system shall allow the user to use a generated password when adding a new account.
BR-004	Keep saved accounts available every time the program is used.	FR-011	The system shall save the accounts list to a file when the user exits the program.
		FR-012	The system shall load the saved accounts from the file when the program starts, or start with an empty list if no file exists.
BR-005	Make the program easy to use through a simple menu.	FR-013	The system shall display a menu of the available options and repeat it until the user chooses Exit.
		FR-014	The system shall display an error message and ask again when the user enters an invalid menu choice or an invalid Y/N answer.

Pseudocode

```
START PasswordManager

  PROCESS declare accounts list, choice, isRunning
  PROCESS set isRunning to TRUE
  IF saved accounts file exists
        PROCESS load saved accounts from file into accounts list
  ELSE
        PROCESS start with an empty accounts list
  END IF

  WHILE isRunning is TRUE

        DISPLAY "===== PASSWORD MANAGER ====="
        DISPLAY "1. Add Account"
        DISPLAY "2. View All Accounts"
        DISPLAY "3. Search Account"
        DISPLAY "4. Update Account"
        DISPLAY "5. Delete Account"
        DISPLAY "6. Generate Random Password"
        DISPLAY "7. Exit"
        INPUT choice

        IF choice is 1
        PROCESS run AddAccount
        ELSE IF choice is 2
        PROCESS run ViewAccounts
        ELSE IF choice is 3
        PROCESS run SearchAccount
        ELSE IF choice is 4
        PROCESS run UpdateAccount
        ELSE IF choice is 5
        PROCESS run DeleteAccount
        ELSE IF choice is 6
        PROCESS run GeneratePassword and get password
        DISPLAY "Generated password: ", password
        ELSE IF choice is 7
        PROCESS save accounts list to file
        DISPLAY "Goodbye!"
        PROCESS set isRunning to FALSE
        ELSE
        DISPLAY "Invalid choice. Please enter a number from 1 to 7."
        END IF

  END WHILE

END PasswordManager
```

```
START AddAccount
  INPUT platform
  WHILE platform is empty
        DISPLAY "Platform name cannot be empty."
        INPUT platform
  END WHILE

  INPUT username
  WHILE username is empty
        DISPLAY "Username cannot be empty."
        INPUT username
  END WHILE

  DISPLAY "1. Enter my own password  2. Generate a random password"
  INPUT option
  WHILE option is not 1 and option is not 2
        DISPLAY "Invalid option. Enter 1 or 2."
        INPUT option
  END WHILE

  IF option is 2
        PROCESS run GeneratePassword and get password
        DISPLAY "Generated password: ", password
  ELSE
        INPUT password
        WHILE password is empty
        DISPLAY "Password cannot be empty."
        INPUT password
        END WHILE
  END IF

  PROCESS add platform, username, and password to accounts list
  DISPLAY "Account saved successfully."
END AddAccount


START ViewAccounts
  IF accounts list is empty
        DISPLAY "No saved accounts."
  ELSE
        PROCESS set number to 1
        WHILE number is not more than the total accounts
        DISPLAY number, platform, username, and password of that account
        PROCESS add 1 to number
        END WHILE
  END IF
END ViewAccounts

START SearchAccount
  INPUT keyword
  PROCESS set found to FALSE
  PROCESS set number to 1
```

```
  WHILE number is not more than the total accounts
        IF platform of that account contains keyword (ignore uppercase and lowercase)
        DISPLAY platform, username, and password of that account
        PROCESS set found to TRUE
        END IF
        PROCESS add 1 to number
  END WHILE
  IF found is FALSE
        DISPLAY "No matching account found."
  END IF
END SearchAccount


START UpdateAccount
  IF accounts list is empty
        DISPLAY "No accounts to update."
  ELSE
        PROCESS run ViewAccounts
        INPUT accountNumber
        IF accountNumber is not a number or is less than 1 or is more than the total accounts
        DISPLAY "Invalid account number."
        ELSE
        INPUT newUsername
        INPUT newPassword
        IF newUsername is not empty
        PROCESS replace username of the selected account
        END IF
        IF newPassword is not empty
        PROCESS replace password of the selected account
        END IF
        DISPLAY "Account updated successfully."
        END IF
  END IF
END UpdateAccount


START DeleteAccount
  IF accounts list is empty
        DISPLAY "No accounts to delete."
  ELSE
        PROCESS run ViewAccounts
        INPUT accountNumber
        IF accountNumber is not a number or is less than 1 or is more than the total accounts
        DISPLAY "Invalid account number."
        ELSE
        DISPLAY "Are you sure you want to delete this account? (Y/N)"
        PROCESS run ReadYesNo and get confirm
        IF confirm is Y
        PROCESS remove the selected account from accounts list
        DISPLAY "Account deleted successfully."
        ELSE
        DISPLAY "Deletion cancelled."
```

```
        END IF
        END IF
  END IF
END DeleteAccount

START GeneratePassword
  DISPLAY "Enter password length (8 to 32):"
  INPUT length
  WHILE length is not a number or is less than 8 or is more than 32
        DISPLAY "Invalid length. Enter a number from 8 to 32."
        INPUT length
  END WHILE

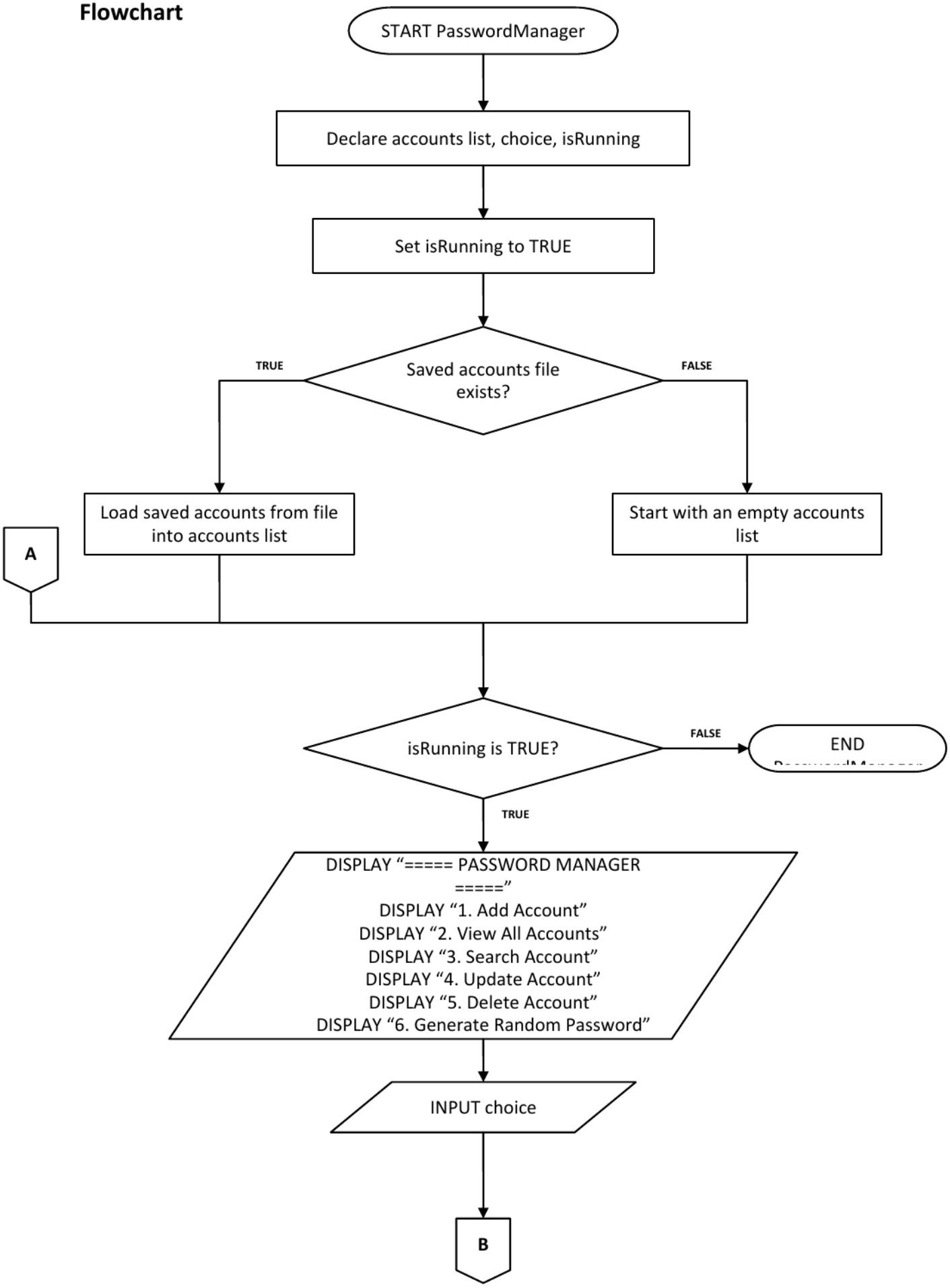
  DISPLAY "Include letters? (Y/N)"
  PROCESS run ReadYesNo and get useLetters
  DISPLAY "Include numbers? (Y/N)"
  PROCESS run ReadYesNo and get useNumbers
  DISPLAY "Include symbols? (Y/N)"
  PROCESS run ReadYesNo and get useSymbols

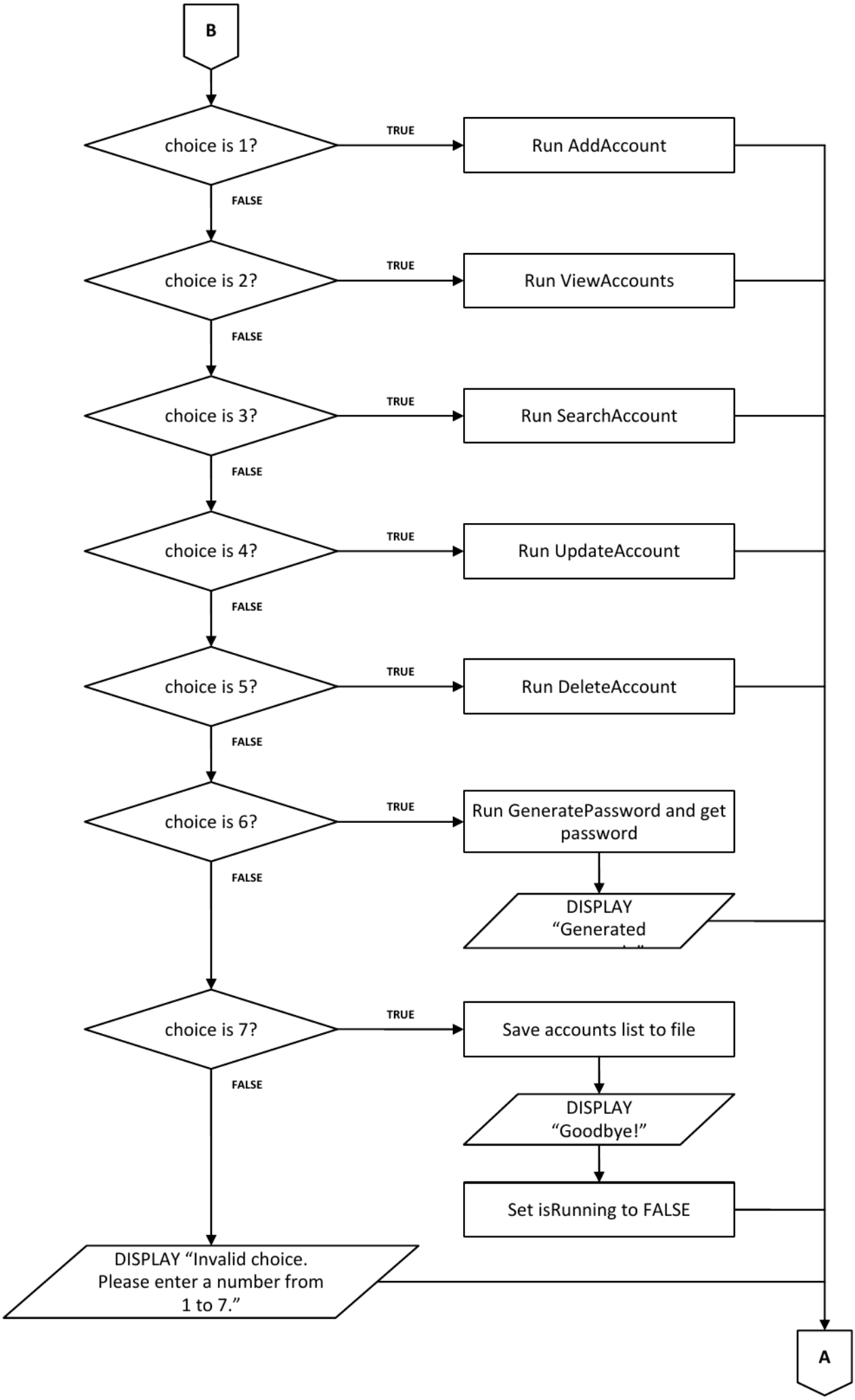
  IF useLetters is N and useNumbers is N and useSymbols is N
        DISPLAY "Select at least one type. Letters will be used."
        PROCESS set useLetters to Y
  END IF

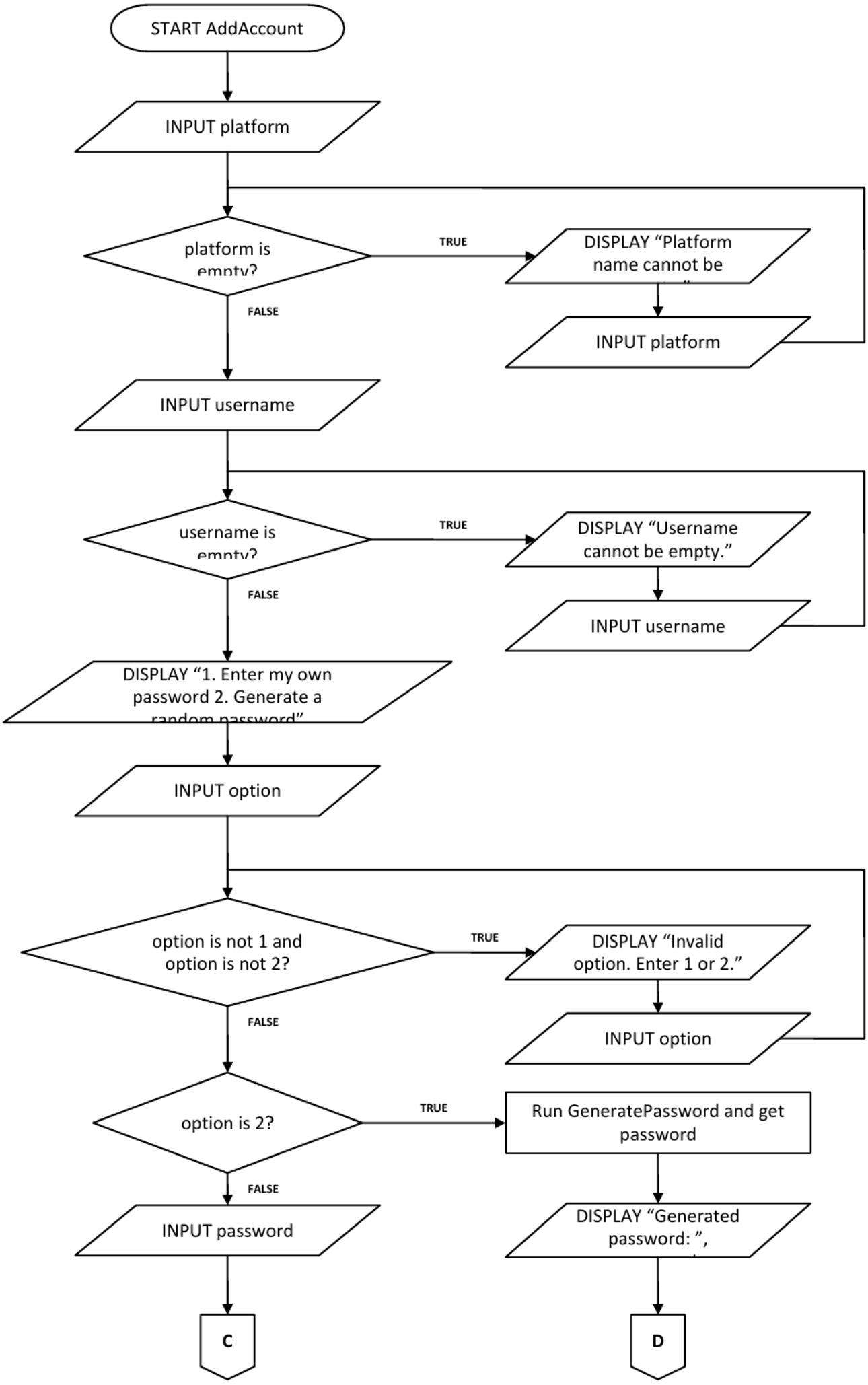
  PROCESS build character pool from the selected types
  PROCESS set password to empty
  PROCESS set count to 1
  WHILE count is not more than length
        PROCESS pick one random character from character pool
        PROCESS add the character to password
        PROCESS add 1 to count
  END WHILE
END GeneratePassword

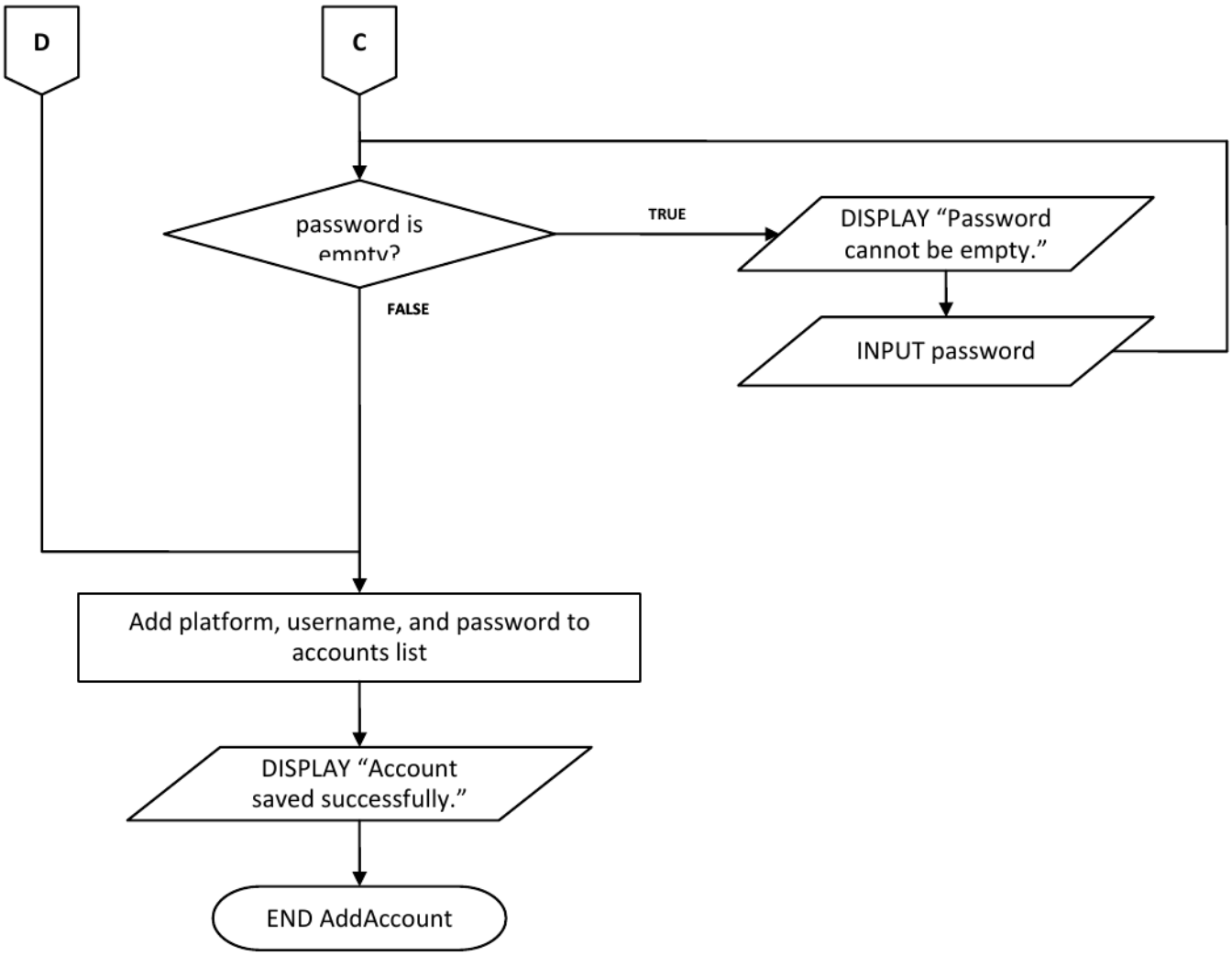

START ReadYesNo
  INPUT answer
  WHILE answer is not Y and answer is not N
        DISPLAY "Invalid answer. Enter Y or N."
        INPUT answer
  END WHILE
END ReadYesNo
```

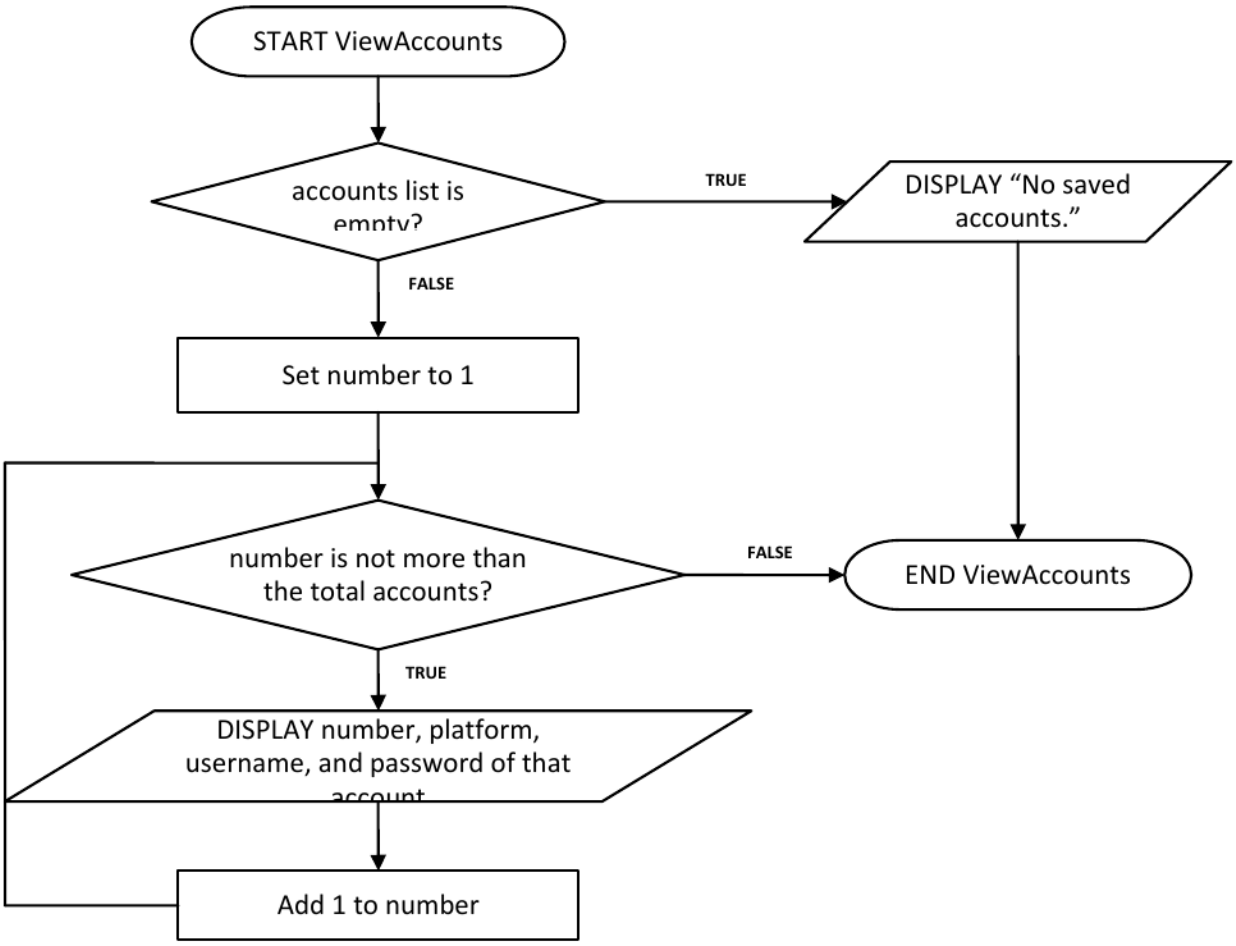
Flowchart

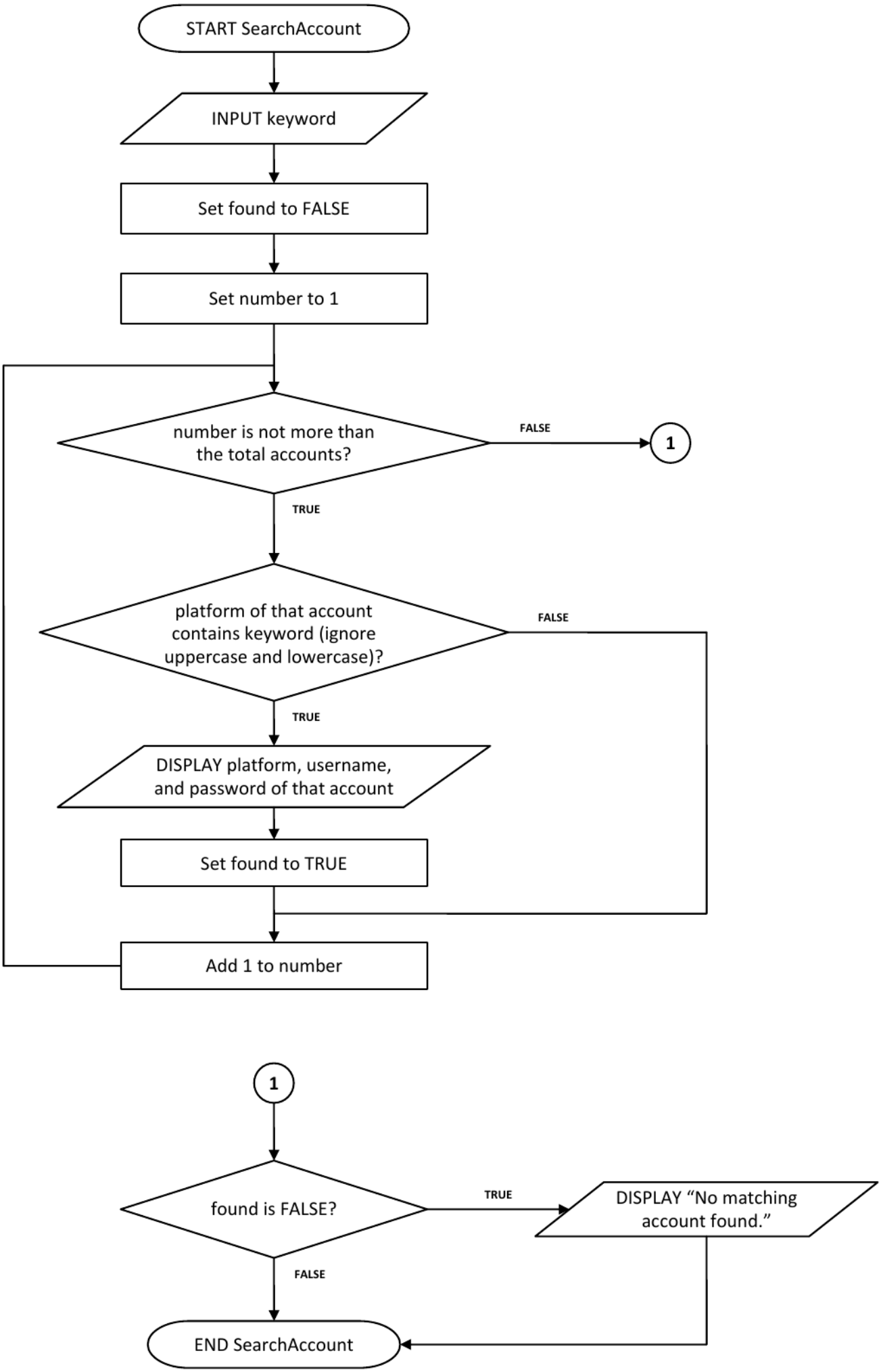

START PasswordManager
Declare accounts list, choice, isRunning
Set isRunning to TRUE
Saved accounts file exists?
TRUE
FALSE
Load saved accounts from file into accounts list
Start with an empty accounts list
A
isRunning is TRUE?
FALSE
END
TRUE
DISPLAY "===== PASSWORD MANAGER =====”
DISPLAY “1. Add Account”
DISPLAY “2. View All Accounts”
DISPLAY “3. Search Account”
DISPLAY “4. Update Account”
DISPLAY “5. Delete Account”
DISPLAY “6. Generate Random Password”
INPUT choice
B

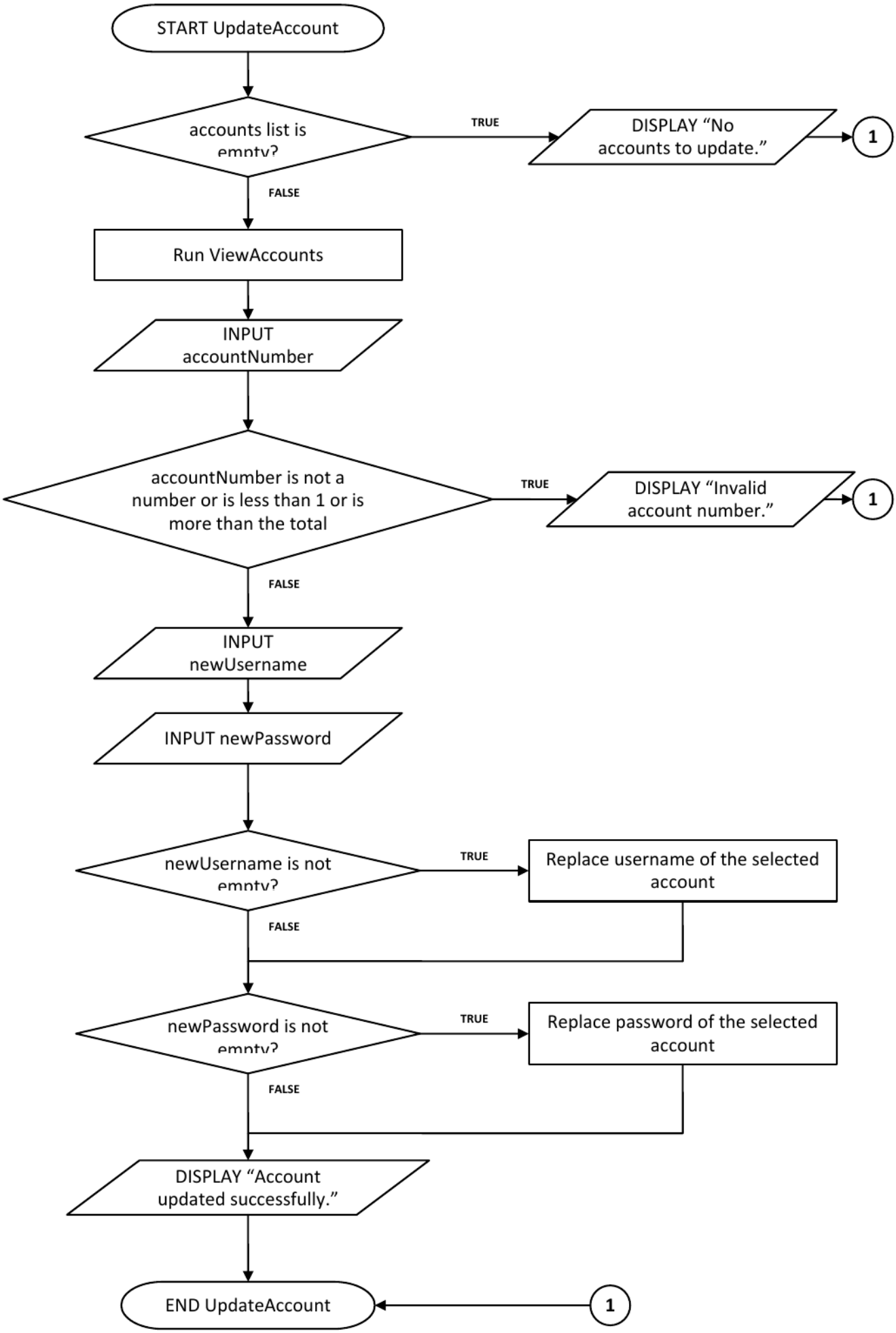

B
choice is 1?
TRUE
Run AddAccount
FALSE
choice is 2?
TRUE
Run ViewAccounts
FALSE
choice is 3?
TRUE
Run SearchAccount
FALSE
choice is 4?
TRUE
Run UpdateAccount
FALSE
choice is 5?
TRUE
Run DeleteAccount
FALSE
choice is 6?
TRUE
Run GeneratePassword and get password
DISPLAY "Generated
FALSE
choice is 7?
TRUE
Save accounts list to file
DISPLAY "Goodbye!"
Set isRunning to FALSE
FALSE
DISPLAY "Invalid choice. Please enter a number from 1 to 7."
A

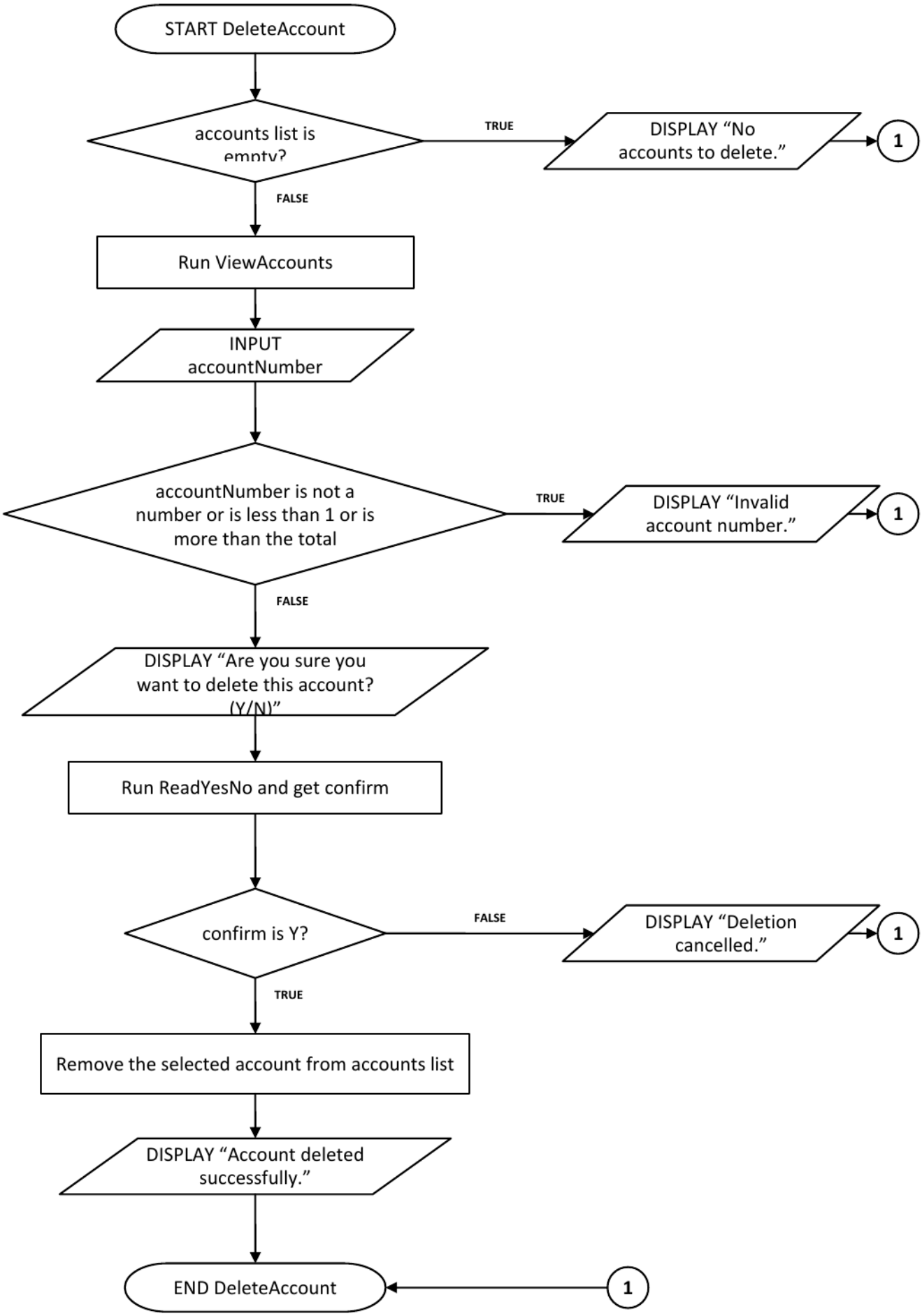

START AddAccount
INPUT platform
platform is empty?
TRUE
DISPLAY "Platform name cannot be
INPUT platform
FALSE
INPUT username
username is empty?
TRUE
DISPLAY "Username cannot be empty."
INPUT username
FALSE
DISPLAY "1. Enter my own password 2. Generate a random password"
INPUT option
option is not 1 and option is not 2?
TRUE
DISPLAY "Invalid option. Enter 1 or 2."
INPUT option
FALSE
option is 2?
TRUE
Run GeneratePassword and get password
FALSE
INPUT password
DISPLAY "Generated password: ",
C
D

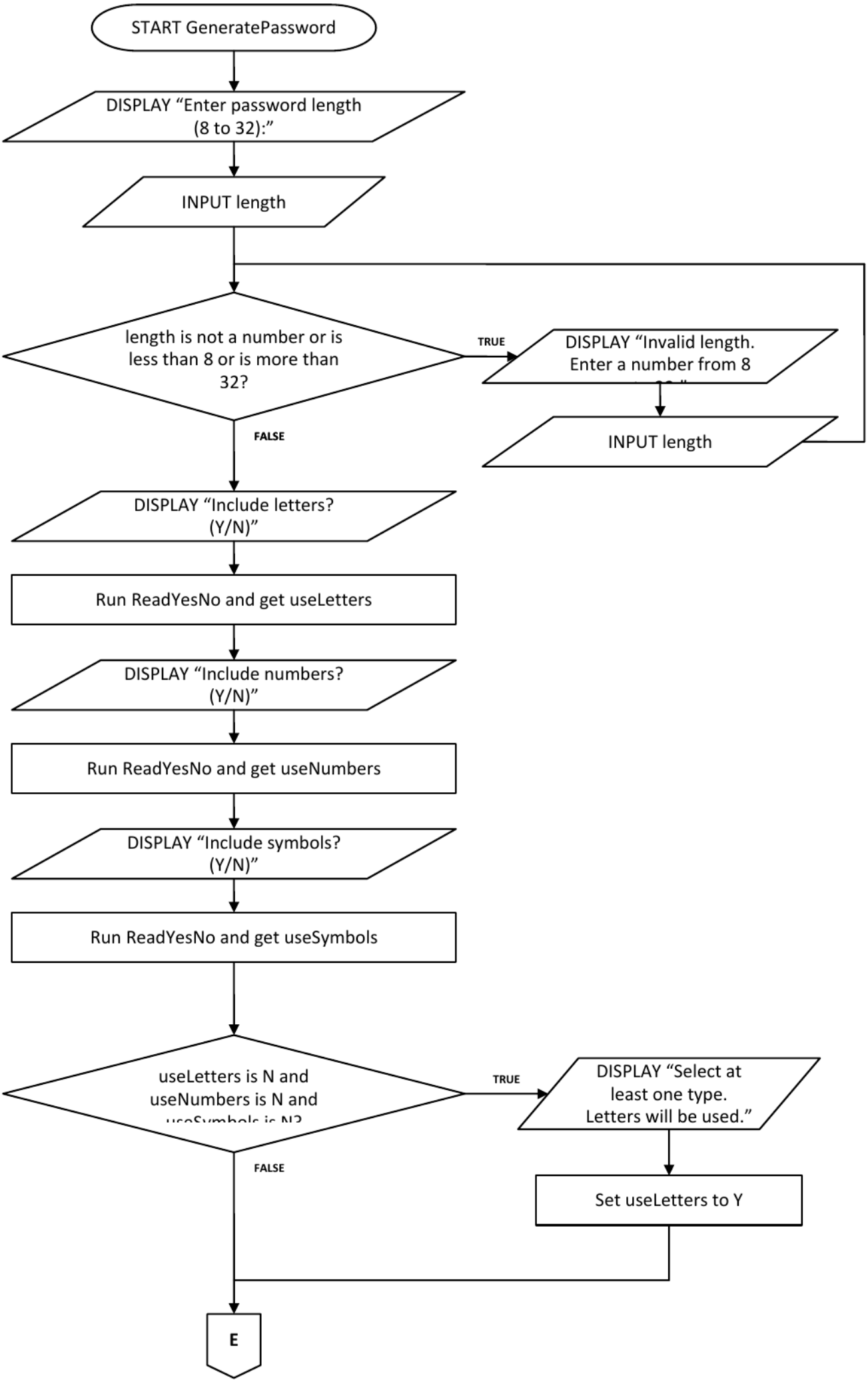

D
C
password is empty?
TRUE
DISPLAY “Password cannot be empty.”
INPUT password
FALSE
Add platform, username, and password to accounts list
DISPLAY “Account saved successfully.”
END AddAccount

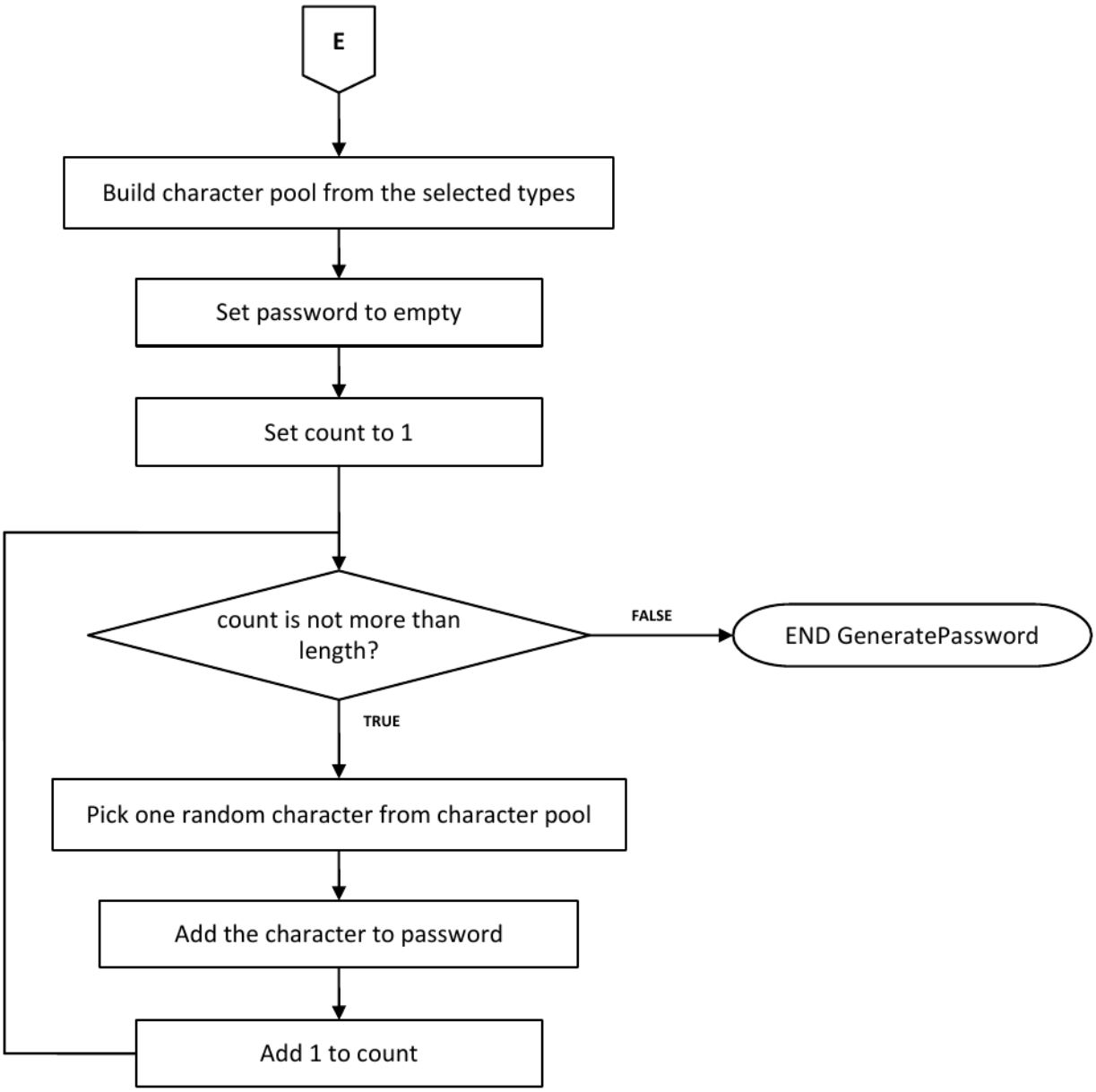

START ViewAccounts
accounts list is empty?
TRUE
DISPLAY “No saved accounts.”
FALSE
Set number to 1
number is not more than the total accounts?
FALSE
END ViewAccounts
TRUE
DISPLAY number, platform, username, and password of that account
Add 1 to number

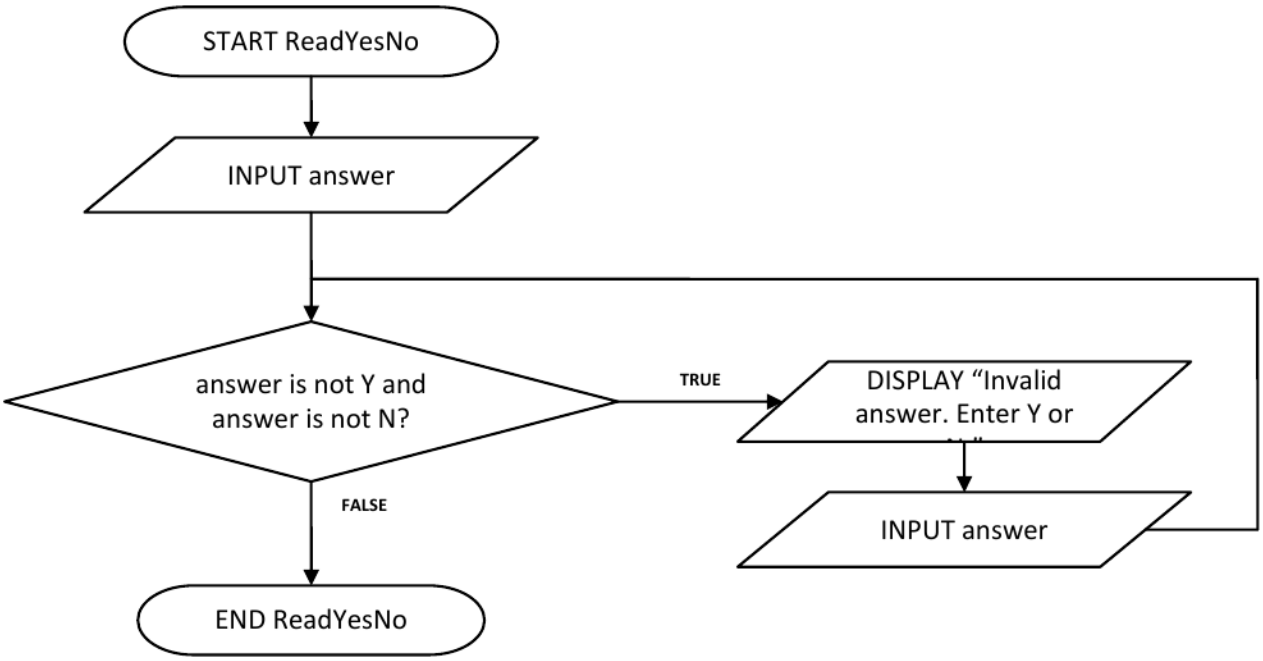

START SearchAccount
INPUT keyword
Set found to FALSE
Set number to 1
number is not more than the total accounts?
FALSE
1
TRUE
platform of that account contains keyword (ignore uppercase and lowercase)?
FALSE
TRUE
DISPLAY platform, username, and password of that account
Set found to TRUE
Add 1 to number
1
found is FALSE?
TRUE
DISPLAY "No matching account found."
FALSE
END SearchAccount


START UpdateAccount
accounts list is empty?
TRUE
DISPLAY “No accounts to update.”
1
FALSE
Run ViewAccounts
INPUT accountNumber
accountNumber is not a number or is less than 1 or is more than the total
TRUE
DISPLAY “Invalid account number.”
1
FALSE
INPUT newUsername
INPUT newPassword
newUsername is not empty?
TRUE
Replace username of the selected account
FALSE
newPassword is not empty?
TRUE
Replace password of the selected account
FALSE
DISPLAY “Account updated successfully.”
END UpdateAccount
1


START DeleteAccount
accounts list is empty?
TRUE
DISPLAY "No accounts to delete."
1
FALSE
Run ViewAccounts
INPUT accountNumber
accountNumber is not a number or is less than 1 or is more than the total
TRUE
DISPLAY "Invalid account number."
1
FALSE
DISPLAY "Are you sure you want to delete this account? (Y/N)"
Run ReadYesNo and get confirm
confirm is Y?
FALSE
DISPLAY "Deletion cancelled."
1
TRUE
Remove the selected account from accounts list
DISPLAY "Account deleted successfully."
END DeleteAccount
1


START GeneratePassword
DISPLAY “Enter password length (8 to 32):”
INPUT length
length is not a number or is less than 8 or is more than 32?
TRUE
DISPLAY “Invalid length. Enter a number from 8
INPUT length
FALSE
DISPLAY “Include letters? (Y/N)”
Run ReadYesNo and get useLetters
DISPLAY “Include numbers? (Y/N)”
Run ReadYesNo and get useNumbers
DISPLAY “Include symbols? (Y/N)”
Run ReadYesNo and get useSymbols
useLetters is N and useNumbers is N and
TRUE
DISPLAY “Select at least one type. Letters will be used.”
Set useLetters to Y
FALSE
E


E
Build character pool from the selected types
Set password to empty
Set count to 1
count is not more than length?
FALSE
END GeneratePassword
TRUE
Pick one random character from character pool
Add the character to password
Add 1 to count


START ReadYesNo
INPUT answer
answer is not Y and answer is not N?
TRUE
DISPLAY “Invalid answer. Enter Y or
INPUT answer
FALSE
END ReadYesNo